

Take-Home: Voice Ingestion Worker

Context

We run realtime voice. Audio reaches us from several places — a telephony bridge, a browser client, an archive of recordings someone wants replayed through the live path — and each of them hands us audio in whatever shape its own stack preferred. Downstream, the number of things that want that audio keeps growing: transcription, recording, analytics, turn detection. None of them should have to know where a given stream came from, and none of them should be able to hurt each other.

You are building the thing in the middle. Treat it as something that will actually run in production: we care about how it behaves operationally, not only whether it produces correct bytes on a good day. Assume both the set of input sources and the set of consumers will keep growing after you hand it over, and assume someone who is not you will be on call for it.

What to build

A worker that ingests live audio in several materially different input forms, normalizes all of it into a single outbound stream carrying Opus with RFC 2198 redundancy, and serves that stream to multiple independent consumers at the same time. It ships as a container image, built through a Makefile.

Hard constraints

- 1 At least three inputs, and three genuinely different ingestion problems. Three files with different extensions is one problem solved three times. At least one input must be live and packetized — arriving over the wire in real time, not read off disk as fast as the disk allows.
- 2 The output is one normalized stream: Opus, with RFC 2198 redundancy. Every input converges on it regardless of what it arrived as.
- 3 Multiple consumers attach to that output concurrently and independently. No single consumer may stall the worker or any other consumer. You must be able to demonstrate this deliberately, not assert it.
- 4 The output must degrade gracefully under packet loss, and you must be able to show it doing so.
- 5 One make target, on a clean checkout, produces the image. Nothing preinstalled, no manual steps in between.
- 6 Where this brief is ambiguous — transport, framing, redundancy depth, backpressure policy, what a late-joining consumer hears, sample-rate handling across mismatched inputs — decide for yourself, write the decision down, and move on. Picking a weak answer knowingly is fine. Not noticing you picked one is not.

Brownie points

- Consumers that attach to the output and transcribe it. Judged on how cleanly they attach and detach while the worker is under load, not on transcription accuracy.

- Measurements from your own machine: added end-to-end latency as a distribution rather than an average, CPU per concurrent stream, and behavior as consumer count rises. Pick the ceiling you test to and say why that number.
- A loss experiment. Induce loss on the output path, show what a consumer receives with your redundancy scheme and without it, and describe the method precisely enough that we could rerun it.
- One thing you got wrong. What you expected, what actually happened, what you changed.
- A decision log, one page, hard limit. Each entry: the decision, the alternative you rejected, and the observation that would make you reverse it.

Scope note

A strong submission is small. Every file in the repository is a file you will be asked to justify out loud. Reach for the smallest thing that satisfies the constraints above and holds up when we push on it.

How this is graded

- Does it work under the conditions you claim — 30%
- Quality of the decisions and the reasoning behind them — 30%
- Evidence: measurements, loss experiment, honesty of the writeup — 25%
- Build and packaging — 15%

There is a 30-minute live session after submission. We will ask you to walk through the code and then change a constraint on the spot — a different redundancy depth, a consumer that wants a different sample rate, an input source that jitters. You will be reasoning about your own design out loud with us. Build accordingly.

Submission Instructions

Once your solution is complete, submit it by sharing access to your GitHub repository. Follow the steps below:

- 1 Push your complete solution — code, Makefile, README, and any supporting materials (decision log, measurements, loss experiment writeup) — to a GitHub repository.
- 2 Ensure the repository README includes clear instructions to build and run the project (the single make target described above), along with any notes needed to reproduce your demonstrations and measurements.
- 3 Grant repository access to **recruitments@trademarkia.com**. If the repository is private, add this email address (or the associated GitHub account) as a collaborator with at least read access. If the repository is public, this step is not required, but the link must still be shared as described below.
- 4 Reply to the assignment email you received, with your GitHub repository link included in the body of the reply. Do not send the link through any other channel unless you are explicitly asked to.
- 5 Double-check, before replying, that the link is reachable by someone outside your organization/account and that access has actually been granted — not just requested.

Access checklist before you reply:

- Repository pushed with final, working code (not a work-in-progress branch).
- README covers build/run steps, decisions, and where to find measurements/loss experiment evidence.
- Access granted to recruitments@trademarkia.com (for private repos).
- Reply sent to the original email with the repository link included.